

COMP0227 Lab 00: Running SLAM and Reading the Displays

5th October, 2026

1 Scope and Purpose

This lab is a sanity check. It has three aims. The first is walk through installing the software on your computer. The second is to confirm that it runs correctly. Third, to provide some indication of the features the software provides. We currently anticipate this software will be a major one which will be used throughout the module for lab and coursework activities.

The software we will use throughout this module is a modified version of pySLAM, a visual SLAM system written in Python and C++. It contains classical and learned features, loop closing, dense reconstruction, depth prediction and semantic mapping. We have made a number of significant modifications, particularly involving the build and deployment system, the c++ core, and PyTorch, to maximise performance of different systems. So far it has been tested on native Linux, windows WSL2 and natively on Apple Silicon Macs. We have also fixed a large number of bugs and issues with system stability. We intend to contact the upstream author to integrate into the main library but given the cadence of changes (more than 250 in less than a week), this will require some unrolling.

pixi uses a set of *levels* and these have been ordered to gradually introduce more capabilities over the duration of the module. Each level will bring in new models, and with them will bring substantial resources. Also, some of the later models we intend to run will be extremely large; we are exploring the use of remote servers to support these activities.

In this lab you will install its *default level*. This has been set up to largely provide the capabilities of ORB-SLAM2. You will be able to run monocular SLAM on a short video which comes with the code, and learn what each of the windows it opens is showing you. Lab 01, which is in the same session, builds on this: you will run on standard datasets and optionally your own datasets from COMP0222, and compare different feature descriptors.

2 What You Need

- One of: Linux (x86-64) with an NVIDIA GPU; a Mac with Apple silicon (macOS 14 or later); or Windows 11 with an NVIDIA GPU, using WSL2 (Ubuntu inside Windows) and then following the Linux instructions.
- About 30 GB of free disk space and an internet connection.
- git.

If your computer has no NVIDIA GPU and is not an Apple silicon Mac, talk to a TA: the software can run on the CPU only, but learned features will be slow. We have some lab laptops available which can be used during the lab session.

3 Activities

Activity 1: Install pixi and Get the Code

pySLAM is installed with `pixi`, a package manager which puts everything the software needs into a folder inside the code (`.pixi`). It does not touch `conda` or the rest of your system, and you do not need to install CUDA or a compiler yourself. Hopefully this will reduce or eliminate python packages going to war with one another, with you being collateral damage.

The installation instructions are provided directly in the repository.

Run all the commands in the rest of this lab from the `pyslam` folder.

Activity 2: Build and Check

Run the following three commands, one after the other:

```
pixi run build
pixi run check
pixi run models
```

The first downloads the environment and builds the C++ modules. This will take the most amount of time (roughly 12 minutes). The second tests that these modules load and that the optimisers work. The third downloads the learned models which we will use in later labs (SuperPoint, LightGlue and CosPlace).

If a command stops, for example because a download breaks, run the same command again: what is already done is skipped.

How do you know that it worked?

- `build` ends with a line which starts `OK:` and says that the native modules share one `pybind11` ABI.
- `check` ends with the `GTSAM` and `g2o` tests passing.
- `models` prints one line per model, with `OK` and the device it ran on. This should be `cuda` (NVIDIA GPU) or `mps` (Apple silicon). If it says `cpu`, your GPU is not being used: tell a TA.

Now run:

```
pixi run doctor
```

This prints one line for each thing pySLAM needs: the machine, the memory, the environment, the GPU, every C++ module and the data. Each line starts with `OK`, `WARN` or `FAIL`. The last line should be:

```
READY: 0 failed, 0 warnings
```

If it is not, read the lines marked `WARN` or `FAIL`: they say what is missing and which command fixes it.

Activity 3: Run SLAM

Run:

```
pixi run slam
```

This runs monocular SLAM, using ORB features, on a video of 1101 frames taken from sequence 06 of the KITTI dataset: a car drives down a street, turns round, and comes back to where it started. The video is played at the frame rate of the camera (10 frames per second), so the run takes about two minutes.

After a few seconds several windows open, mostly on top of one other. A lot of text is also printed in the terminal; you can ignore it for now.

At the end of the video, the video frame view will freeze and the terminal will say that the sequence has ended. The windows will all stay open until you close the program (press q or ESC on the frame view window). Leave them open for the next activity.

Activity 4: Read the Displays

The windows are how pySLAM tells you what it is doing. Find each of the windows below, and use them to answer the questions.

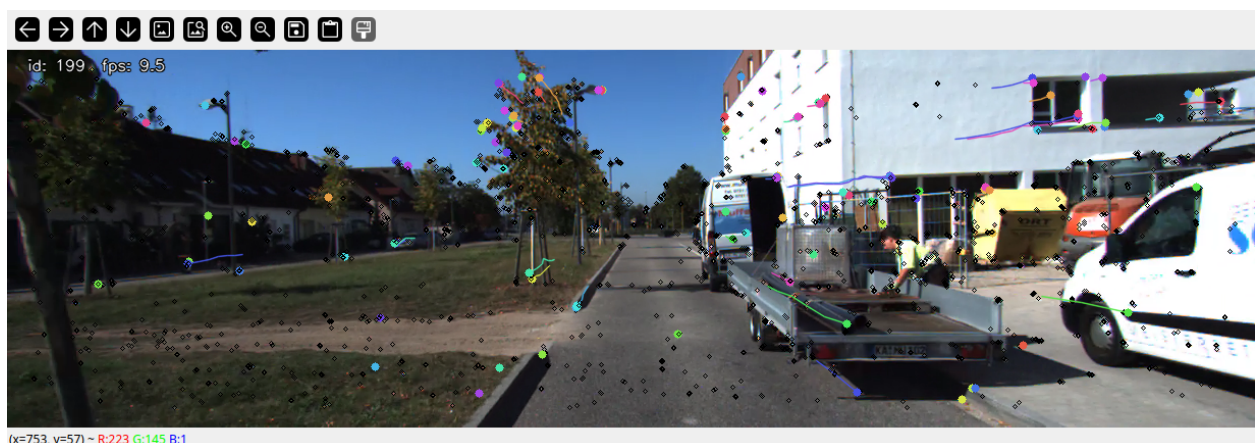


Figure 1: The Camera window.

Camera

The camera window, shown in Figure 1, shows the current frame, with its number in the top left corner. Each small black circle is a feature which was found in the image but which was not matched to a point in the map. Each coloured dot is a feature which is matched to a map point; the line attached to it shows where that point was seen in the previous frames.

- Roughly what proportion of the features are matched to map points?
- In which parts of the image are the matched features? Where are there none? Why?
- In which direction do the lines point? What does this tell you about how the camera is moving?

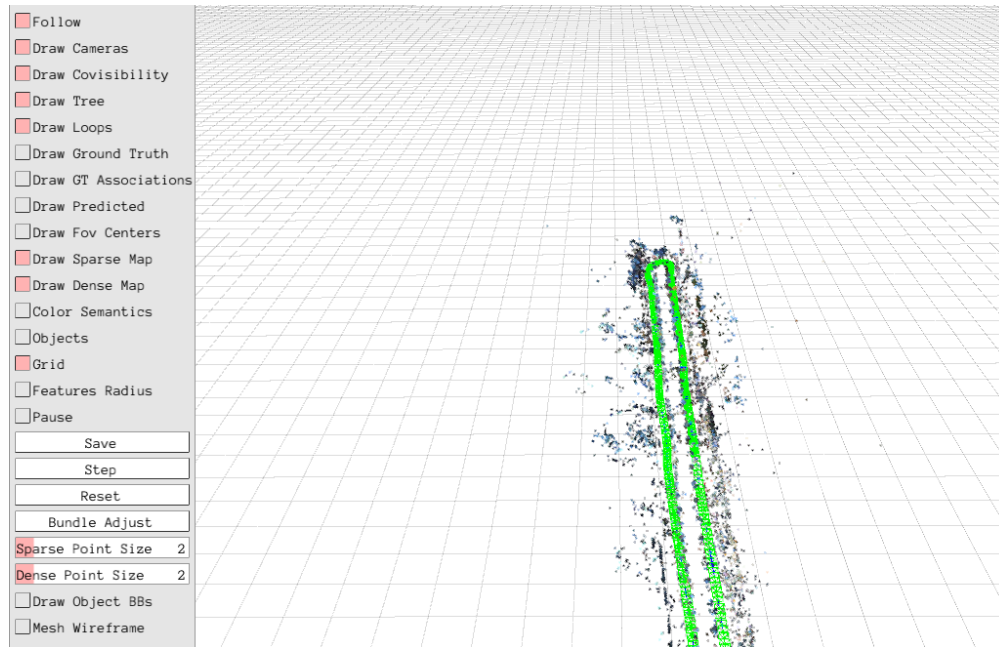


Figure 2: The Map Viewer window, near the end of the run.

Map Viewer

The Map Viewer (Figure 2), is very similar to the view you will have seen in ORB-SLAM. The map is shown in 3D. The small dots are the map points. The green shapes are the cameras of the keyframes, joined by lines to the other keyframes which see the same points. The blue shape is the current camera. You can move and rotate the view with the mouse buttons, and zoom with the wheel. Untick **Follow** if you want the view to stay where you put it.

- Can you recognise the street, and the turn at the end of it, from the map points?
- Tick **Draw Ground Truth**. How close is the estimated trajectory to the true one? Where is it worst?
- The video is from a single camera. What does this mean for the scale of the map?

Diagnostic Plots

There are four diagnostic plots (Figure 3) which provide more detail about what's going on inside pySLAM. Each is named after what it shows.

- **# matches:** for each frame, how many features were matched and how many of these are points in the map.
- **mean chi2 error:** how well the estimated camera pose and the map agree with the measurements in the images.
- **timing:** how long each part of the system took, in seconds. Tracking runs for every frame; local mapping, loop detection and loop closing run in the background.
- **trajectories:** gt vs (aligned)estimated: the estimated and the true position of the camera over time, and the error between them.

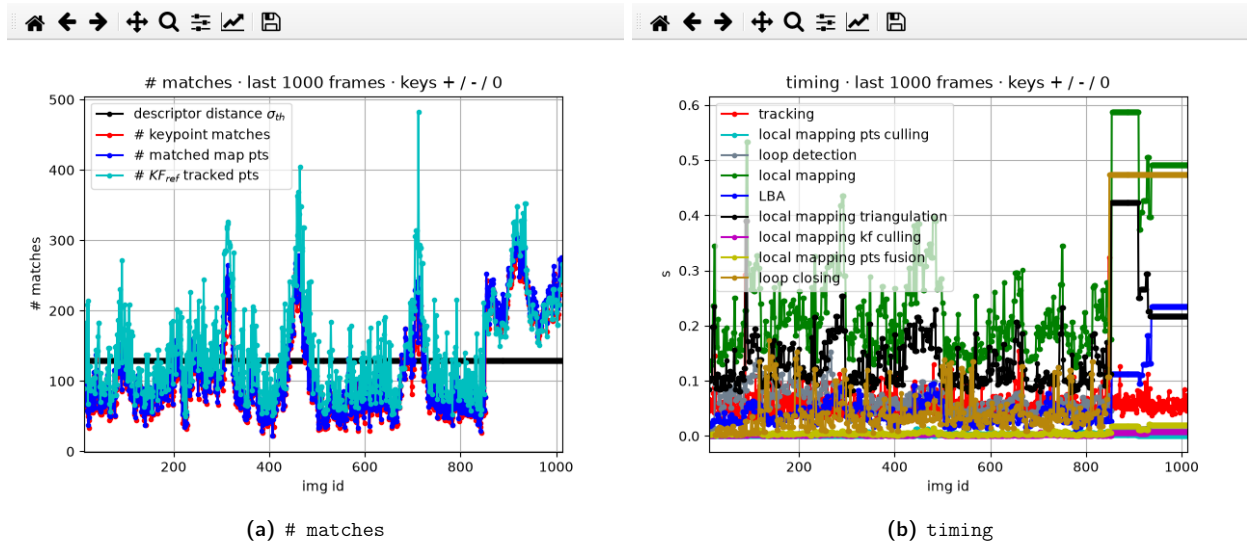


Figure 3: Two of the plot windows.

The first three plots show sliding windows of 1000 frames. The window length can be changed. Click on a plot and press + to zoom in, - to zoom out and 0 to show the whole run.

- The camera produces a frame every 0.1 seconds. Does tracking keep up with it on your computer?
- Look at the number of matches at about frame 850. What happened? (The next two windows will help.)

Loop Closing

Loop closing (Figure 4) looks for places the camera has imaged before. When it finds one, observation edges can be linked between to the earlier keyframe, reducing drift and increasing the accuracy of the map. pySLAM uses similar logic to ORB-SLAM: loop closure candidates are generated based on visual similarity between keyframes using a visual bag-of-words.

- loop closing: similarity matrix: the entry in row i and column j shows how similar keyframe i looks to keyframe j . The brighter it is, the more similar they are.
- loop closing: consistent candidates: the current keyframe at the top and, below it, the earlier keyframes which loop closing thinks show the same place.
- Why is the diagonal of the similarity matrix bright?
- Near the end of the run, bright entries appear in two corners of the matrix. What do they mean?
- Are the candidates in the other window really the same place as the current keyframe? Look at the ones from early in the run as well as the ones from the end.
- What changed in the Map Viewer when the loop was closed?

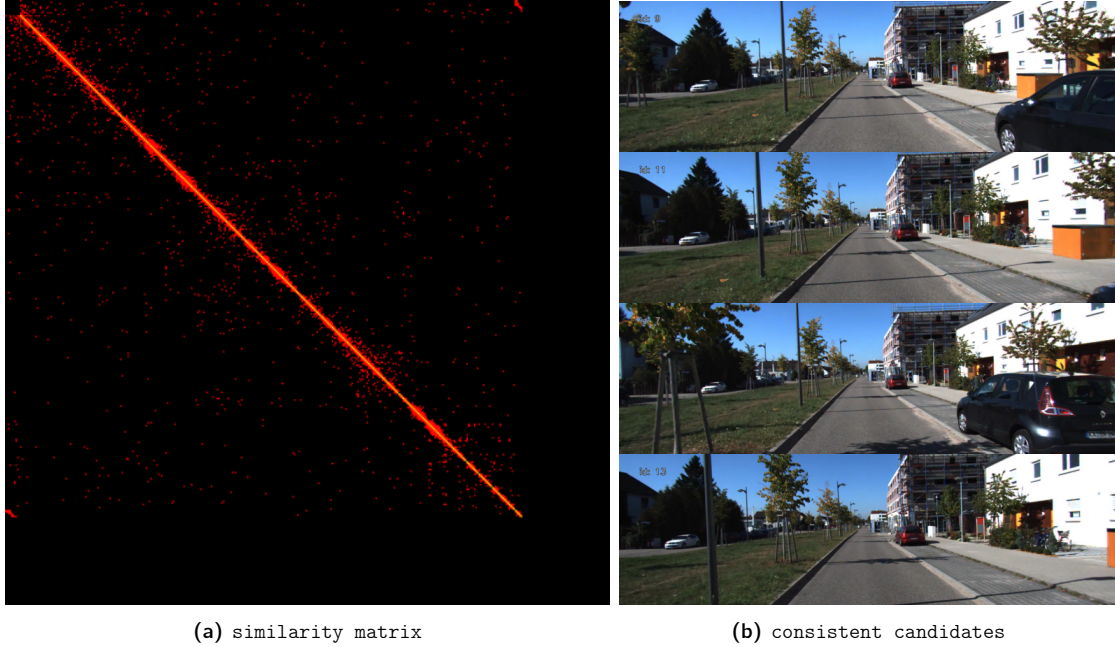


Figure 4: The two loop closing windows, near the end of the run.

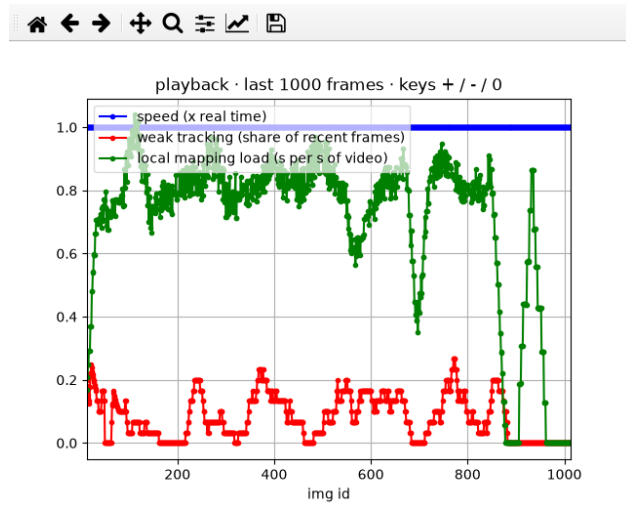


Figure 5: The Playback window.

Playback

The playback window (Figure 5) provides information about the rate at which the video is being played back. We have modified pySLAM to include a throttling mechanism. Normal SLAM systems have to run at real-time, which means that if a frame cannot be processed in time, it must be dropped. For teaching, where there is a huge variability in available resources, this can have a huge impact on tracking performance. We have modified pySLAM to have a mechanism to dynamically change playback speed to match performance with the fastest machines we have tested on.

- **speed ($\times$ real time):** how fast are the frames being delivered to pySLAM? Smaller is slower. 1.0 means full rate of video, 0.5 means half rate, etc. You should see it drop and rise occasionally as more challenging areas are detected.
- **weak tracking:** this is the signal used to control the playback mechanism. A sliding window of 30 frames are considered. Each window is classified as being weakly tracked if less than 50% of the features in the tracked keyframe can be mapped to the current frame. It is a signal that a new keyframe is required, and hence the SLAM system will be undertaking additional work.
- **local mapping load:** this is the ratio of the amount of time taken by the local mapper when processing the last 30 frames, divided by the amount of time for 30 frames to arrive. It is a direct measure of how real-time the system is running: a value greater than 1.0 means that it cannot keep up with the full frame rate. Real-time performance is enforced in this case by scaling down the rate of keyframe delivery.

The plot shown is for a fast machine which can easily keep up the frame rate.

Activity 5: Controls

Run `pixi run slam` again, and this time try the controls in the panel on the left of the Map Viewer:

- **Pause** stops the video; untick it to continue. While it is paused, Step processes one frame. When you pause, the trajectory error so far is printed in the terminal.
- **Draw Cameras**, **Draw Covisibility**, **Draw Tree** and **Draw Loops** show or hide the keyframes and the graphs which join them. Find out what each one draws.
- **Features Radius** changes how the features are drawn in the Camera window: the size of each circle is the scale at which the feature was found.

To quit, click on the Camera window and press `q` or `Esc`. The windows close, the trajectory error is computed, and the results are written to a folder `results/metrics_<date>`.

Activity 6: The Trajectory Error

Close any other programs which are using a lot of the processor, and run:

```
pixi run doctor --run
```

This repeats the checks of Activity 2 and then runs SLAM on the same video, without the windows. The last check should look like this:

```
OK    main_slam.py --headless: 1101/1101 frames, 0 lost (0.0%), 151 s,
      ATE RMSE 15.4 m
```

The software passes if all the frames were processed, none (or very few) were lost, and the last line says `READY`. The number at the end is the absolute trajectory error (ATE): the root mean square distance between the estimated and the true positions of the camera, after the estimated trajectory has been scaled and aligned with the true one. We will look at it in detail later in the module.

Now run the same command again, and compare the two errors.

- Are they the same? On our computers the error on this video has been anywhere between about 11 and 21 m.

- The video and the software are exactly the same in both runs. What could make the results different?
- What does this mean for how you should compare two methods?
- What do you expect to happen if you run two copies at the same time?

4 If Something Goes Wrong

- Run the command again. Downloads continue where they stopped, and what is already built is skipped.
- Read the message. When something is not installed or not built, pySLAM says what it is and which command installs or builds it.
- Look at `docs/TEACHING_SETUP.md` and `docs/TROUBLESHOOTING.md` in the code. They cover, among other things, the memory needed for the build and the windows under WSL2.
- If tracking is lost at the same places in every run, your computer may be too slow for the frame rate of the camera. Try `pixi run slam --throttle`, which slows the video down when tracking gets weak.
- Ask a TA. Please have ready the command you ran, the whole of the error message, and the output of `pixi run doctor`.